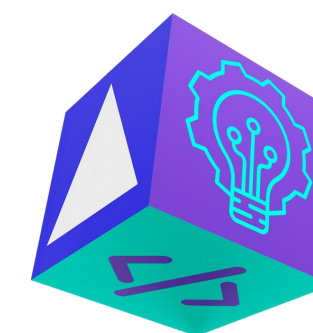
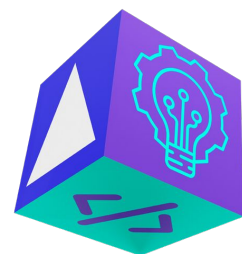




RESIDÊNCIA
EM SOFTWARE
BAHIA + TECNOLOGIA + EMPREENDEDORISMO

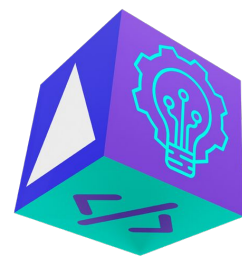
Trilha

**Desenvolvimento Front-End e
Conceitos de User Interface e
User Experience (UI/UX)**



Consumo de API

1. O Que É Uma API
2. O Que É REST
3. O Que É Uma API RESTful
4. Métodos HTTP
5. Função Fetch

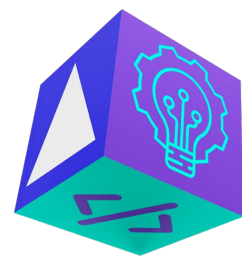


O que é uma API

Application Programming Interface (Interface de Programação de Aplicações) é uma maneira de um ou mais programas computacionais ou componentes de comunicação se comunicarem um com o outro.

A API define um conjunto de rotinas e padrões estabelecidos por uma aplicação para que outras aplicações possam utilizar suas funcionalidades.

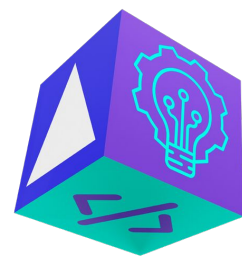
As APIs permitem que diferentes sistemas se comuniquem entre si, promovendo a integração e a interoperabilidade.



O que é uma API

Importância das APIs

- Facilita a integração entre sistemas.
- Permite a reutilização de funcionalidades.
- Promove a modularidade e a flexibilidade no desenvolvimento de software.

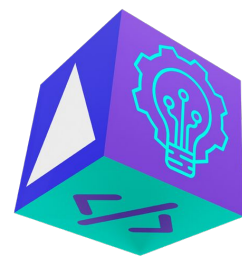


O que é REST

REST (Representational State Transfer) é um estilo de arquitetura para sistemas distribuídos.

Foi introduzido por Roy Fielding em sua tese de doutorado em 2000.

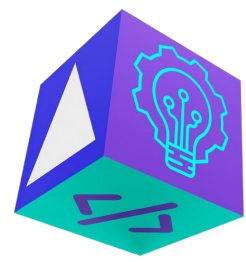
REST utiliza os padrões e protocolos da web, principalmente o HTTP, para criar APIs.



O que é REST

Princípios do REST

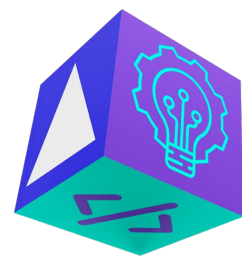
- **Cliente-Servidor:** A interface do usuário deve ser separada do armazenamento de dados, permitindo a portabilidade do código entre diferentes plataformas.
- **Stateless:** Cada requisição do cliente para o servidor deve conter todas as informações necessárias para entender e processar o pedido.
- **Cacheable:** As respostas devem definir se são ou não cacheáveis, para evitar que clientes enviem múltiplas requisições idênticas.



O que é REST

Princípios do REST

- **Interface Uniforme:** Simplifica e desacopla a arquitetura, permitindo que cada parte evolua independentemente.
- **Sistema em Camadas:** A arquitetura pode ser composta por camadas hierárquicas, com cada uma funcionando de forma independente.



O que é uma API RESTful

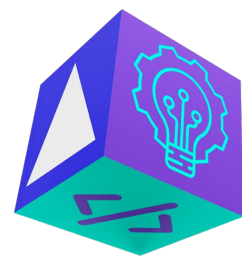
Uma API RESTful é uma API que adere aos princípios de design REST.

Ela usa os métodos HTTP para realizar operações CRUD (Create, Read, Update, Delete) nos recursos da web.

Recursos e URI

- **Recursos:** Tudo em uma API RESTful é considerado um recurso. Por exemplo, usuários, produtos, pedidos, etc.
- **URI (Uniform Resource Identifier):** É a forma como os recursos são identificados. Cada recurso possui uma URI única.

Ex.: `https://api.exemplo.com/usuarios`



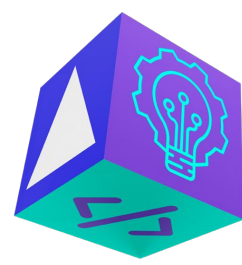
Métodos HTTP

Métodos Principais

- **GET:** Recupera informações de um recurso.
- **POST:** Cria um novo recurso.
- **PUT:** Atualiza um recurso existente.
- **DELETE:** Remove um recurso.

Exemplos

- **GET /usuarios:** Obtém a lista de usuários.
- **POST /usuarios:** Cria um novo usuário.
- **PUT /usuarios/1:** Atualiza o usuário com ID 1.
- **DELETE /usuarios/1:** Remove o usuário com ID 1.



Métodos HTTP

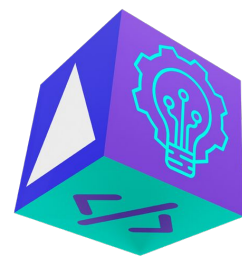
Estrutura de uma Requisição HTTP

Componentes

- **URL:** O endereço do recurso.
- **Método HTTP:** O tipo de operação a ser realizada.
- **Cabeçalhos:** Metadados sobre a requisição (e.g., Content-Type, Authorization).
- **Corpo:** Dados a serem enviados na requisição (usado em POST e PUT).

Exemplo de Requisição Post

```
POST /usuarios HTTP/1.1
Host: api.exemplo.com
Content-Type: application/json
{
  "nome" : "João",
  "email" : "joao@example.com"
}
```



Métodos HTTP

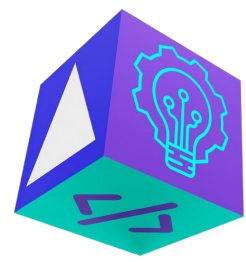
Estrutura de uma Resposta HTTP

Componentes

- **Status Code:** Código de status da resposta (e.g., 200 OK, 404 Not Found).
- **Cabeçalhos:** Metadados sobre a resposta.
- **Corpo:** Dados retornados pela API.

Exemplo de Resposta

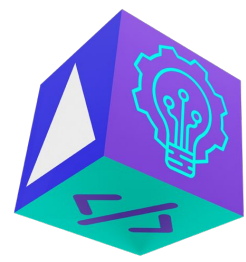
```
HTTP/1.1 201 Created
Content-Type: application/json
{
  "id" : 1,
  "nome" : "João",
  "email" : "joao@example.com"
}
```



Métodos HTTP

Principais Códigos de Status

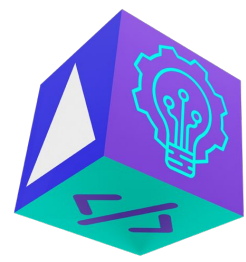
- **200 OK:** Requisição bem-sucedida.
- **201 Created:** Recurso criado com sucesso.
- **400 Bad Request:** Requisição malformada.
- **401 Unauthorized:** Falta de autenticação.
- **403 Forbidden:** Falta de permissão.
- **404 Not Found:** Recurso não encontrado.
- **500 Internal Server Error:** Erro no servidor.



Métodos HTTP

Benefícios das APIs RESTful

- **Simplicidade:** Uso dos padrões e protocolos HTTP torna mais fácil o desenvolvimento.
- **Escalabilidade:** Separação entre cliente e servidor permite melhor escalabilidade.
- **Performance:** Cacheabilidade melhora a performance.
- **Flexibilidade:** Interface uniforme permite a evolução independente das partes.



Função Fetch

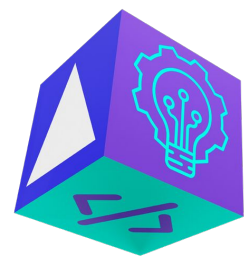
A função fetch é uma API nativa do JavaScript que fornece uma interface para fazer requisições HTTP.

Ela é baseada em Promises, o que facilita o uso de operações assíncronas, como obter dados de um servidor.

A função fetch é bastante flexível e pode ser usada para realizar vários tipos de requisições HTTP, como GET, POST, PUT, PATCH e DELETE.

Ex. de requisição GET com fetch:

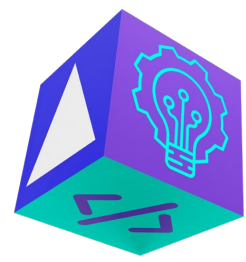
```
const data = await fetch(`http://localhost:3000/tasks`);
```



Função Fetch

Exemplo de implementação de uma requisição GET em uma aplicação Angular:

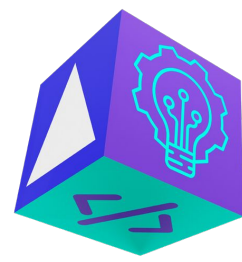
```
@Component ({  
  ...  
})  
  
export class AppComponent {  
  private baseUrl = 'http://localhost:3000';  
  
  async getTaks(): Promise<Task[]> {  
    const data = await fetch(`${this.baseUrl}/tasks`);  
    return (await data.json()) ?? [];  
  }  
}
```



Função Fetch

Exemplo de implementação de uma requisição GET com passagem de parâmetro em uma aplicação Angular:

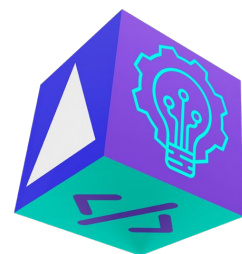
```
@Component ({  
  ...  
})  
  
export class AppComponent {  
  private baseUrl = 'http://localhost:3000';  
  async getTask(id: number) {  
    const data = await fetch(`${this.baseUrl}/tasks/${id}`);  
    return (await data.json()) ?? {};  
  }  
}
```



Função Fetch

Exemplo de implementação de uma requisição POST em uma aplicação Angular:

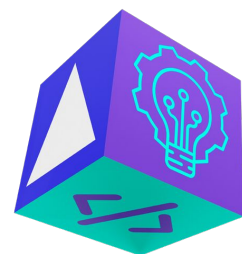
```
async addTask(taskTitle: string) {  
  const data = await fetch(`${this.baseUrl}/tasks`, {  
    method: 'POST',  
    headers: {  
      'Content-Type': 'application/json',  
    },  
    body: JSON.stringify({ title: taskTitle, completed: false }),  
  });  
  return await data.json();  
}
```

Função Fetch

Exemplo de implementação de uma requisição PUT em uma aplicação Angular:

```
async updateStatus(id: number, status: boolean) {  
  const data = await fetch(`${this.baseUrl}/tasks/${id}`, {  
    method: 'PUT',  
    headers: {  
      'Content-Type': 'application/json',  
    },  
    body: JSON.stringify(status),  
  });  
  return await data.json();  
}
```



Função Fetch

Exemplo de implementação de uma requisição DELETE em uma aplicação Angular:

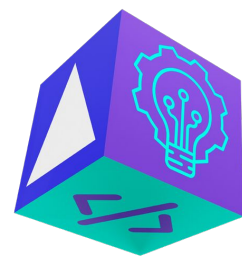
```
async deleteTask(id: number) {  
  const data = await fetch(`${this.baseUrl}/tasks/${id}`, {  
    method: 'DELETE',  
  });  
  return await data.json();  
}
```




Função Fetch

Para tratar os possíveis erros das requisições, deve-se usar o try catch:

```
async getTaks(): Promise<Task[]> {  
  try {  
    const data = await fetch(`${this.baseUrl}/taskss`);  
    this.error = `${data.status}`;  
    return (await data.json()) ?? [];  
  } catch (error) {  
    this.error = 'Erro ao buscar os dados. Cód.: ' + this.error;  
    throw error;  
  }  
}
```



Referências

- <https://angular.dev>
- <https://aws.amazon.com/pt/what-is/restful-api/>
- <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Methods><https://www.ibm.com/br-pt/topics/rest-apis>

Irlan Moreira

irlan.moreira@ifrn.edu.br

Wagner

irlan.moreira@ifrn.edu.br

